

PROJEKT ENGINEERING

Implementierung

Herwig Mayr

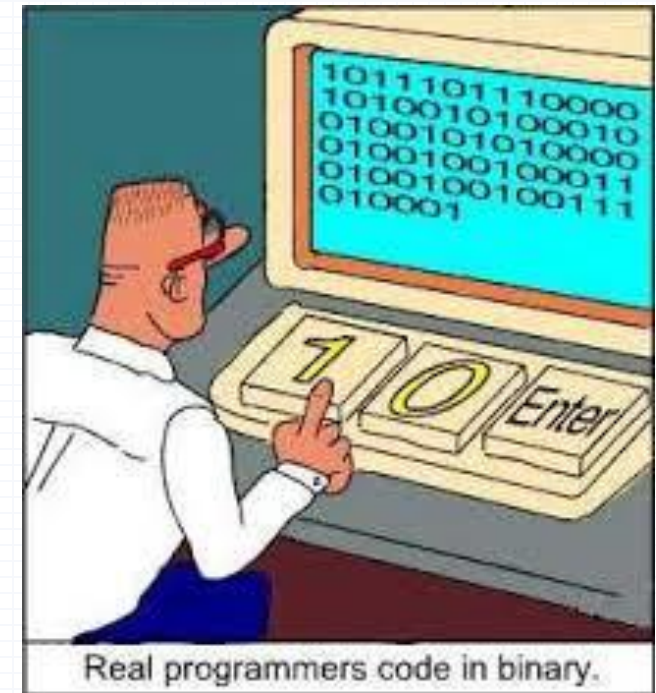
Fakultät für Informatik, Kommunikation und Medien
Fachhochschule OÖ, Hagenberg

Codieren

Codieren Ziel

Codieren – Ziel

1. Umsetzen der Designergebnisse in **Programmcode (Quellcode)**
-> Codieren des Designs soll möglichst klar strukturiert und in einfachen Schritten möglich sein!
2. Übersetzen (lassen) des Quellcodes in geeigneten Code für das Zielsystem



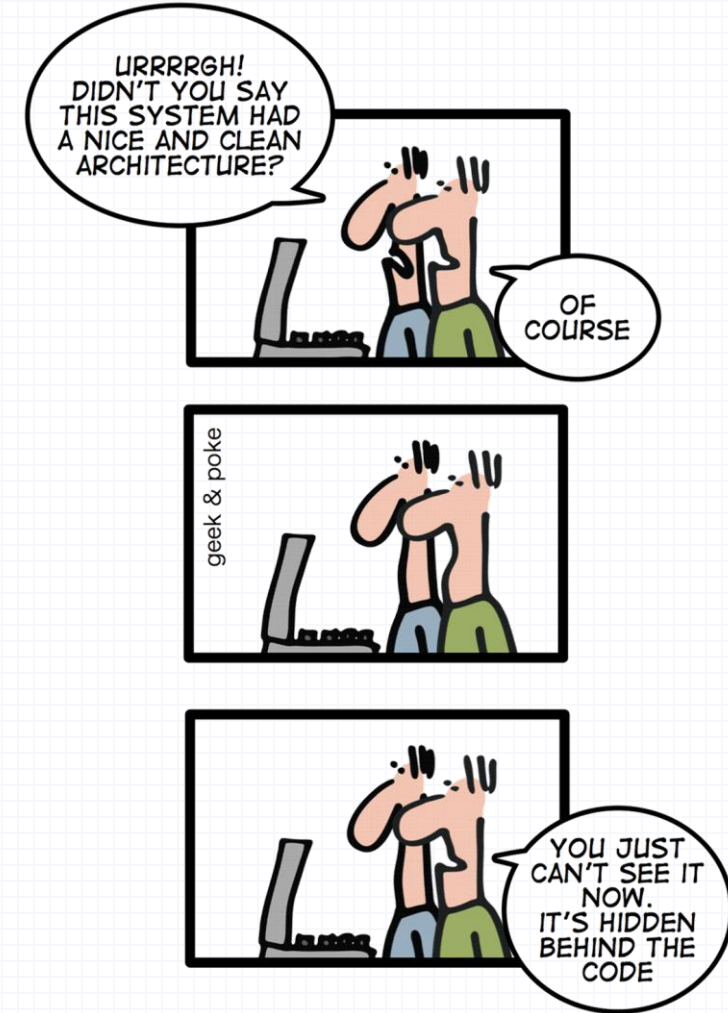
(Grafik: www.motor-talk.de)

Codieren Aufgaben

Codieren – Aufgaben

Schrittweises, korrektes Umsetzen des Designs:

1. Umsetzen des Systemdesigns
2. Umsetzen des Komponentendesigns
3. (komponentenweises) Übersetzen des Quellcodes in Zielcode
4. Einzeltest jeder Komponente
5. Zusammenbau zum Gesamtsystem



(Grafik: geekandpoke.typepad.com)

Codieren – Geschichte der Programmierung

Grundlegende Entwicklungsschritte der Programmierung:

- um 1940 große Maschinennähe (Assembler)
- um 1960 Unterprogramme (erster Abstraktionsschritt)
- um 1970 strukturierte Programmierung
- um 1980 modulatororientierte Programmierung
- um 1990 objektorientierte Programmierung
- um 2000 komponentenorientierte Programmierung
- um 2010 agentenorientierte Programmierung
- um 2020 KI-gestützte Programmierung

(Bild: www.technikimbuerro.at)



Codieren Techniken

Codieren – Techniken

- Transformieren des Designs
- Einfügen logischer Bedingungen
- Instrumentieren des Codes
- Kontinuierliche Integration
- Code-Restrukturierung („Refactoring“)
- Erstellen von Testrahmen

Techniken – Transformieren des Designs

Transformieren des Designs

klassische Kernaufgabe der Programmierung

-> Literatur, z. B.



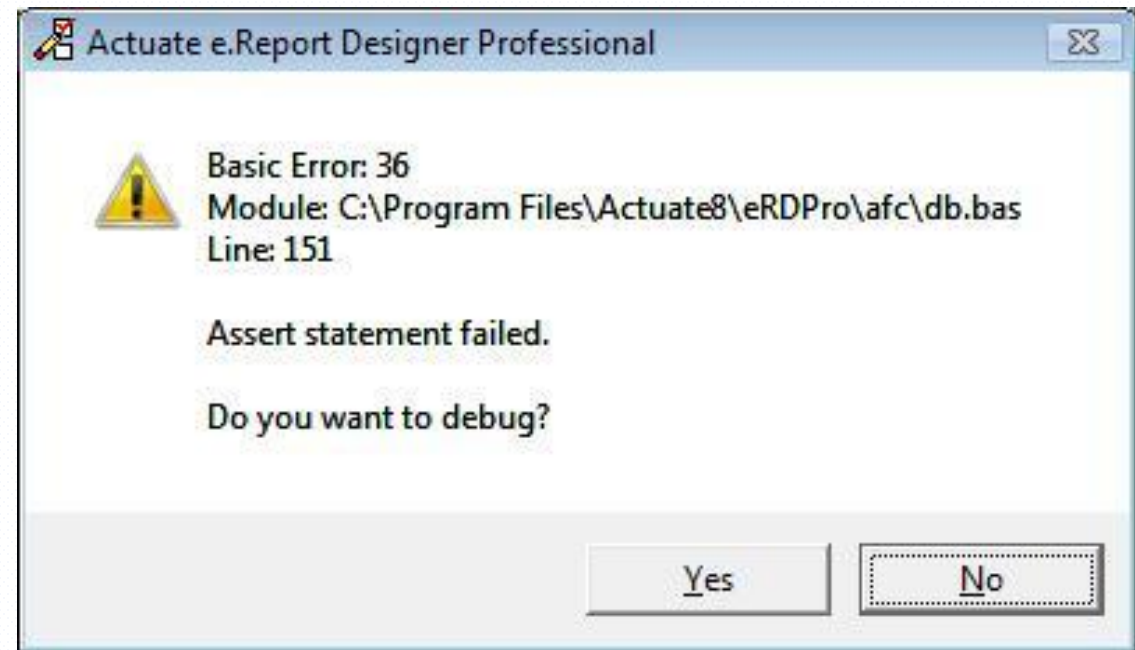
(Grafik: maximotimes.com)

Techniken – Einfügen logischer Bedingungen

Einfügen logischer Bedingungen

Assertionen sind logische Aussagen, die Bedingungen im Programm entsprechen:

- Vorbedingungen
- Nachbedingungen
- Invarianten



Beispiel/Übung: Assertionen (Bedingungen in Programmen)

- Vorbedingungen:
- Nachbedingungen:
- Invarianten:

Techniken – Instrumentieren des Codes

Instrumentieren des Codes

Versehen des Codes mit zusätzlichen Codeteilen zum

- Feststellen der Codelokalität zur Laufzeit
- Prüfen der Pfadabdeckung
- dynamischen Analysieren (Profiling)
- Prüfen der Testqualität (Bebuggen, Mutieren)

Hauptprobleme:

- neue Codeteile können zusätzliche Fehler enthalten
- neue Codeteile verändern manchmal das Programmverhalten

Techniken – Kontinuierliche Integration

engl. „**continuous integration**“ (CI): Jeder Entwickler pflegt kleine Codeänderungen laufend ein (früher „**daily build**“, jetzt „**continuous build**“).

Voraussetzungen:

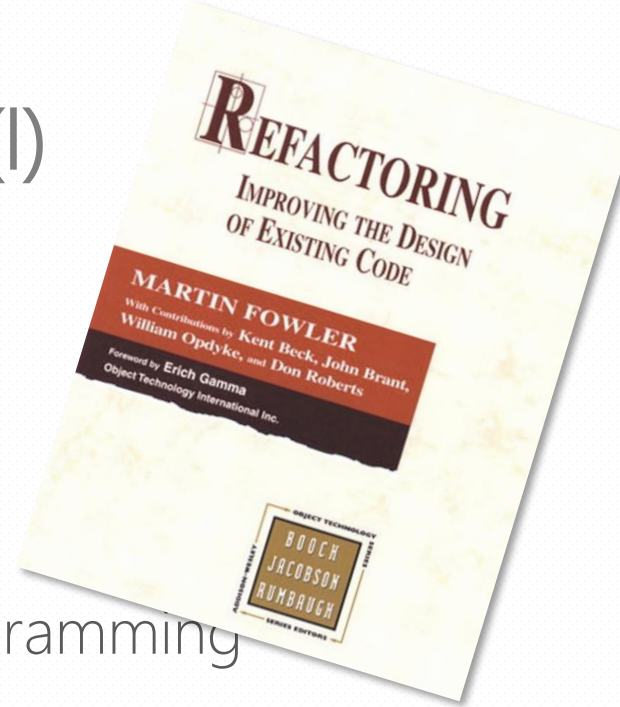
- eine gemeinsame Codebasis
- Übersetzen und Binden auf Knopfdruck
- funktionale Tests hoch automatisiert
- neueste Programmversion laufend zugänglich

-> In der Praxis nur Werkzeug-gestützt sinnvoll möglich (z.B. Apache Ant, CruiseControl, Jenkins)!



Techniken – Code-Restrukturierung („Refactoring“) (I)

Code-Restrukturierung = „disziplinierte Änderung eines existierenden Codes zur Designverbesserung ohne Verhaltensänderung“ [Fowler]



- stammt aus Smalltalk (~1980); ab ~2000 eng mit eXtreme Programming (und später anderen agilen Vorgehensmethoden) verknüpft
- Erkennen restrukturierungsbedürftigen Codes durch Muster („**bad smells**“)
- **positiv**: einleuchtende Muster mit detaillierten Vorschlägen zur Restrukturierung (teilweise sogar automatisierbar, vgl. z.B. Eclipse)
- **negativ**: Kommentare werden auch als Zeichen eines schlechten Codes gewertet! (eher missverständlicher Zweck der Kommentierung)

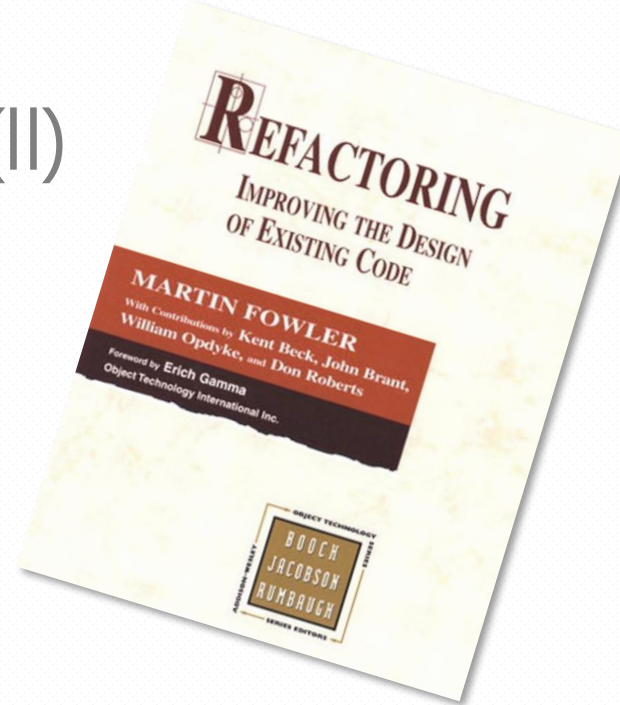
Beispiel/Übung: Ursachen restrukturierungsbedürftigen Codes

Martin Fowler, DER Guru des Refactoring,
stellt dazu einen Katalog online zur Verfügung:
<https://refactoring.com/catalog/>.

Techniken – Code-Restrukturierung („Refactoring“) (II)

Restrukturierung darf Lauffähigkeit nicht verändern!

- nur lauffähige Programme restrukturieren
- automatisierte Tests verwenden (Test-Driven Development)



Weiteres Problem: Bei unerfahrenen Programmierern steigt Restrukturierungsaufwand überproportional! [Boehm]

Restrukturierung kann Architekturprobleme nicht lösen!

-> sinnvollste Anwendung zur **Verbesserung** des Komponentendesigns

Techniken – Erstellen von Testrahmen

Zweck: Erstellen temporärer Programm(teil)e und Daten zu Testzwecken

- Programmierumfang kann umfangreich sein (bis zu gleich viel wie der zu testende Code)
- Code von Testrahmen nicht löschen, sondern nur ausblenden

Komponenten von Testrahmen:

- **Stumpf** („stub“): Testcode für gerufene Routine
- **Treiber** („driver“): Testcode für rufende Routine
- **Attrappe** („mock-up“, „mock object“): Testcode für Komponenten („mock-up“ wird oft für nichtausführbare Benutzerschnittstellen-Prototypen verwendet)
- **Dummydatei** („dummy file“): Datei mit Testdaten

Einschub: Testgesteuerte Entwicklung

Testgesteuerte Entwicklung – Idee

Idee (engl. **Test-driven Development – TDD**): Testfälle werden vor dem entsprechenden Programm(teil) erstellt (eigentlich programmiert)!

Zweistufiges Vorgehen:

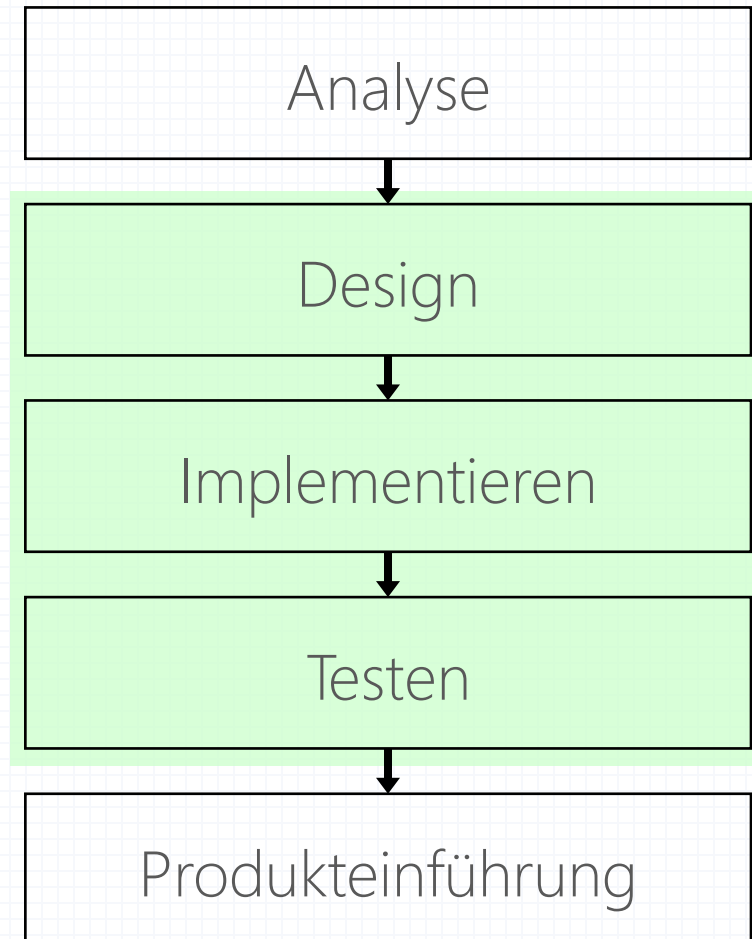
1. Erstellen von Systemtests für Anwendungsfälle (idealerweise durch AG)
2. Erstellen von daraus abgeleiteten Komponententests (durch AN)

In der iterativ-inkrementellen Entwicklung oft
Ersatz für genaue Produktspezifikation:

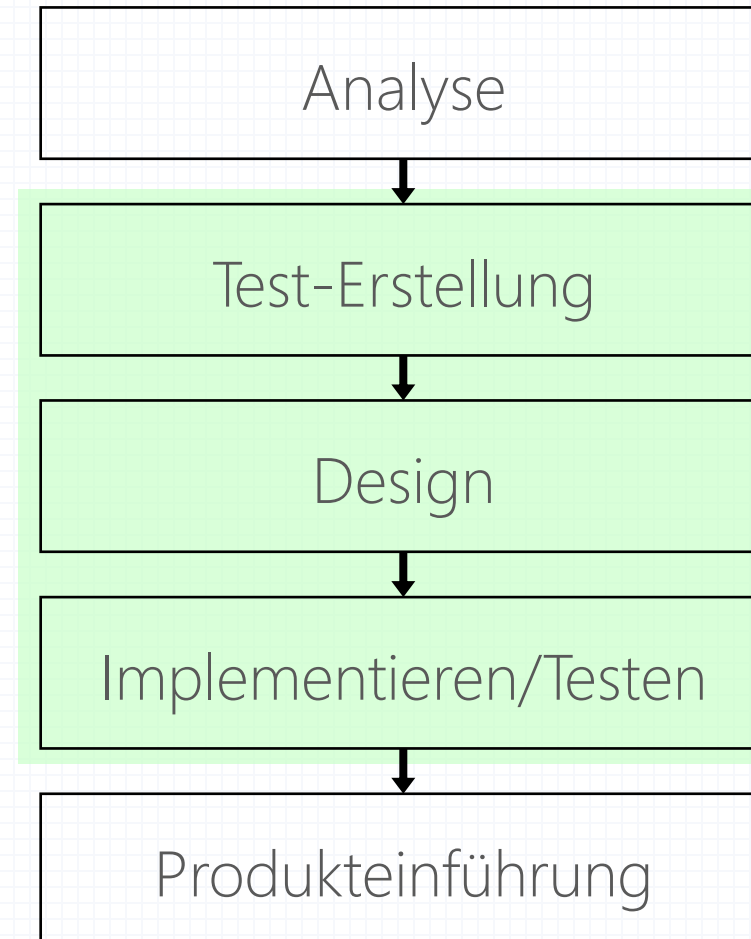
- Produkt erfüllt immer genau die spezifizierten Testfälle.
- Jeder neue Testfall führt zu einer (möglichst kleinen) Produkterweiterung (ggf. Code-Restrukturierung).

Testgesteuerte Entwicklung – Ablauf

Traditionelle Entwicklung



Testgesteuerte Entwicklung



Testgesteuerte Entwicklung – Konsequenz

Testgesteuerte Entwicklung beeinflusst Art des Designs und der Implementierung und ist daher *keine* Technik des Testens!

-> Big-Picture Design notwendig

TDD
ALL CODE IS GUILTY
UNTIL PROVEN INNOCENT

Testgesteuerte Entwicklung ist eine Entwicklungsmethode!



(Bild: www.pathfindersolutions.com)

Beispiel/Übung: Testgesteuerte Entwicklung

■ MyMath.java

```
package at.scch.examples;

public class MyMath {

    public static int maximum(int x, int y) {
        if (x > y) {
            return x;
        } else {
            return y;
        }
    }
}
```

(Beispielcode: R. Ramlar, SCCH Hagenberg)

■ MyMathTest.java

```
package at.scch.examples.tests;

import junit.framework.TestCase;
import at.scch.examples.*;

public class MyMathTest extends TestCase {

    public void testMaximum() {
        int max = MyMath.maximum(4, 3);
        assertEquals(4, max);
    }
}
```

1: Executed object

2: Input values

3: Expected result

==

4: Actual result

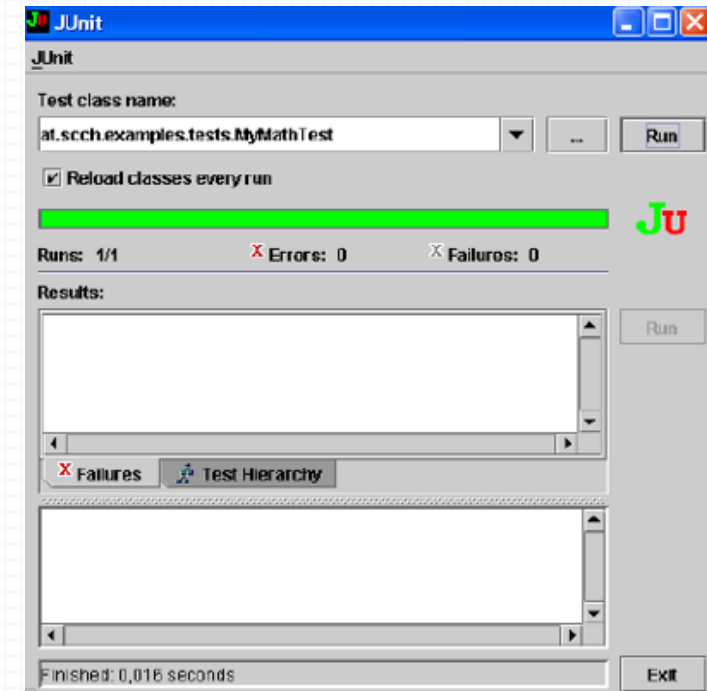
Testgesteuerte Entwicklung – Vorgehen

Testfälle werden jeweils **zu Beginn der Entwicklung** eines Bausteins bzw. Inkrements („Features“ aus **Kundensicht**) erstellt.

Strategien:

1. Starte nicht mit Objekten, starte mit Tests!
2. Erstelle Systemtests für Anwendungsfälle ...
3. ... und daraus Komponententests für dein Design!
4. Führe die Tests laufend (automatisch) aus!
5. Mache kleine Inkremente. Bewahre den Überblick!
6. Verfahre nach „Red-Green-Refactor“ (*Unit)!

Testfälle werden jeweils auch **während der Entwicklung** auf Basis des Big-Picture-Designs (aus **Entwicklersicht**) erstellt.



Testgesteuerte Entwicklung – Vorteile

- + Tests sind **automatisch ausführbar** und daher leicht wiederholbar.
- + Tests werden vor der Realisierung erstellt -> Erzeugung **testbaren Codes**.
- + Tests dienen als **Spezifikation und Dokumentation** („besser als nichts“).
- + Die Implementierung wird in jedem Inkrement getestet (**einfaches Regressionstesten**).
- + Code-Änderungen sind (auch lokal) rasch überprüfbar.
- + Fehler sind rasch lokalisierbar; Folgefehler rasch erkennbar.
- + Fehler bei der Code-Restrukturierung werden normalerweise rasch erkannt.
- + Man verliert **weniger Zeit mit Debuggen**.
- + Zeit für die Testfallentwicklung wird zu Beginn investiert.
Damit fällt eine Belastung zu Iterationsende weg. ;-)

Testgesteuerte Entwicklung – Herausforderungen

- Testfallerstellung und -auswahl werden **nicht** unterstützt.
- Testfallqualität ist schwer beurteilbar.
- **Testfälle hängen** teils voneinander und auch **von** (geänderten) **Anforderungen** ab. Notwendige Änderungen an Testfällen sind schwer erkennbar.
- Eine große Anzahl von Einzeltests **ersetzen nicht** Integrations- und **Systemtests**.
- Techniken der systematischen Testfallerstellung werden oft vernachlässigt.
- „Einige“ korrekt verlaufene Testfälle „sichern“ die Programm-Korrektheit!



Testen kann nur die Anwesenheit von Fehlern zeigen,
aber nicht deren Abwesenheit! [Dijkstra]



Testgesteuerte Entwicklung – Weitere Erkenntnisse (optimistisch)

- + Testklassen sind gleichzeitig Testimplementierung und -dokumentation der zu implementierenden Klassen.
- + Testklassen können als Ersatz für eine nicht vorhandene Spezifikation dienen.
- + Tester und Entwickler arbeiten enger zusammen.
- + Es gibt keinen Code ohne Test.
- + Testklassen liefern Sicherheit bei der Code-Restrukturierung.
- + Man entwickelt nicht zu viel und nicht zu wenig Code auf einmal.



Testgesteuerte Entwicklung – Weitere Erkenntnisse (pessimistisch)

- Die Implementierung wird eher nach hinten verschoben.
- Man versucht, durch viele, einfache Tests die unangenehmen, komplizierten Tests zu ersetzen.
- Man glaubt, dass ein Testfall, der einmal passt, auch immer passt.
- Eine große Anzahl von Einzeltests ersetzt nicht Integrations- und Systemtests.
- Testgesteuerte Entwicklung verleitet zu Einstellungen wie „Warum denn weiter testen, wenn es eh schon läuft?“.
- Größtes Problem: Wer testet die Tests?

Testfallqualität ⇒ Testqualität ⇒ Softwarequalität!

Codieren Werkzeuge

Codieren – Werkzeuge

Entwicklungsumgebungen, auch Programmierumgebungen genannt

für Codieren benötigt:

- Editor
- Compiler / Linker
- Versions- und Konfigurationsverwaltungssystem



Codieren Ergebnisse

Codieren – Ergebnisse

Ergebnisse

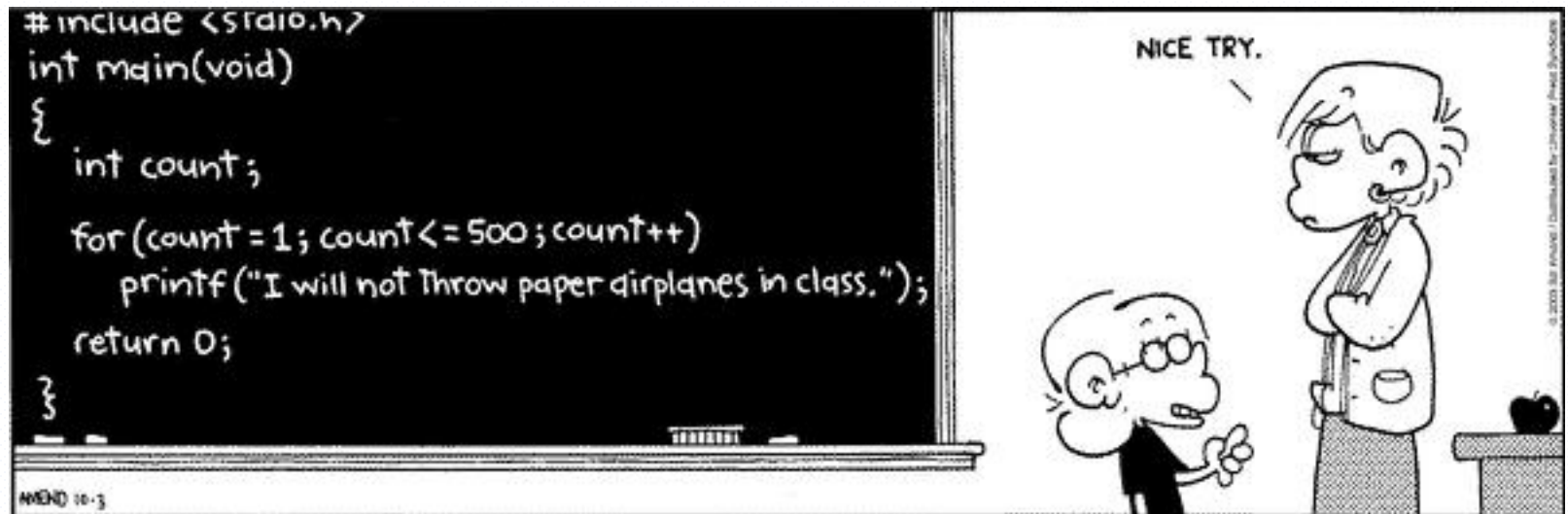
- Quellcode
- Systemdokumentation

Ergebnisse – Quellcode

Quellcode

- Gestaltung muss bestimmten Regeln unterworfen werden
- Kommentierung muss Design widerspiegeln

(Grafik: stackoverflow.com)



Tipp: Möglichst früh entscheiden, ob Quellcode dem Auftraggeber übergeben wird!

Beispiel/Übung: Was ändert sich, wenn der Code dem Auftraggeber übergeben wird?

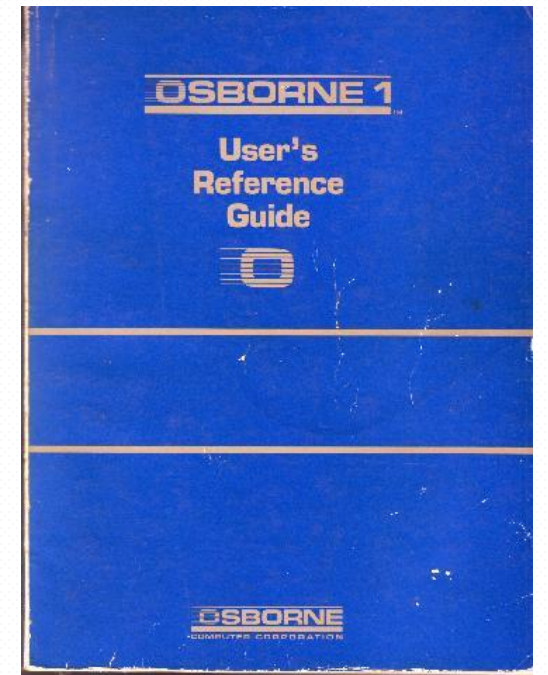
Ergebnisse – Systemdokumentation (I)

Systemdokumentation („Software/System Reference Manual“)

Zweck:

- Erklärung der Implementierung
- Dokumentation des Know-hows
- Grundlage für Wartung
- Unterstützung der Kommunikation im Entwicklerteam (zusätzlich Entwicklerkollegen als Kontrollleser)

(Grafik: wikipedia.org)



Ergebnisse – Systemdokumentation (II)

Systemdokumentation („Software/System Reference Manual“)

Komponenten:

- Beschreibung der Systemstruktur
- Beschreibung des dynamischen Verhaltens
- Beschreibung der Komponenten
- Beschreibung der verwendeten externen Datenhaltung
- Beschreibung der zentralen Begriffe
- Beschreibung der Installation (Detaillierung des Installationshandbuchs)



(Grafik: www.boschrexroth-us.com)

Die Systemdokumentation darf **nicht nur Ergebnisse**, sondern muss **auch die Gründe** dafür (**WARUM**) dokumentieren.

Debuggen

Debuggen Ziel

Debuggen – Ziel

Finden der Ursachen von Fehlern und Mängeln inkl. deren Behebung

meist vom Codeersteller selbst durchgeführt



(Grafik: www.dilbert.com)

Debuggen Aufgaben

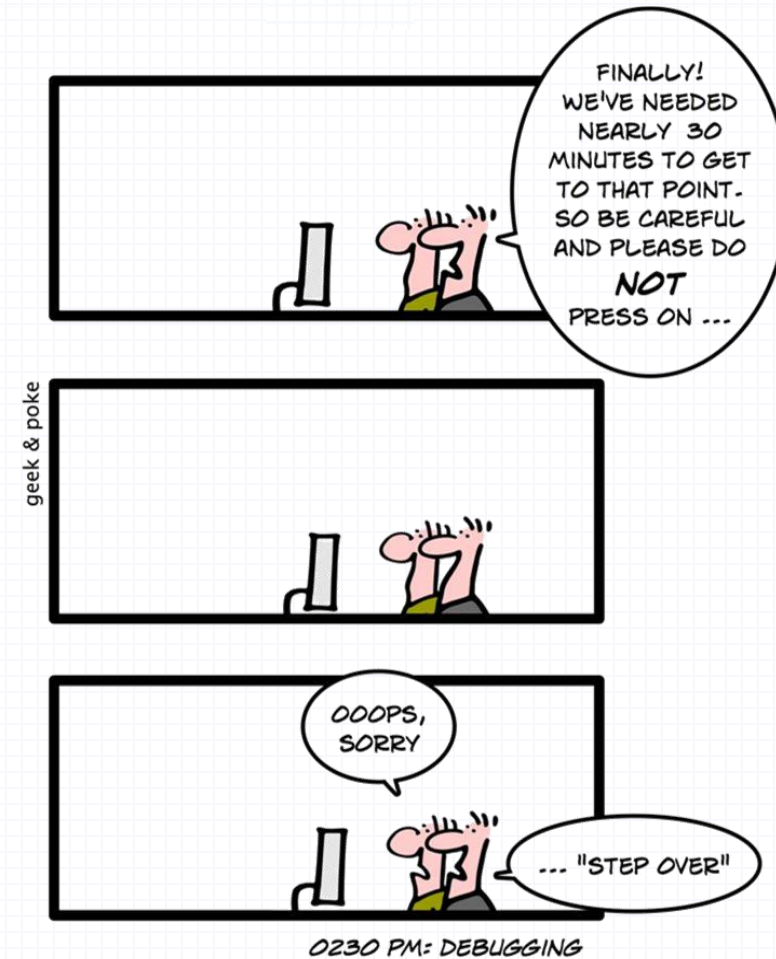
Debuggen – Aufgaben

Sechs Schritte für **effizientes Debuggen**:

1. Fehler stabilisieren
2. Fehler lokalisieren
3. Korrektur ausarbeiten
4. Korrektur durchführen u. dokumentieren
5. Korrektur testen
6. auf ähnliche Fehler prüfen

Unterschiede beim (Blackbox-)Testen:

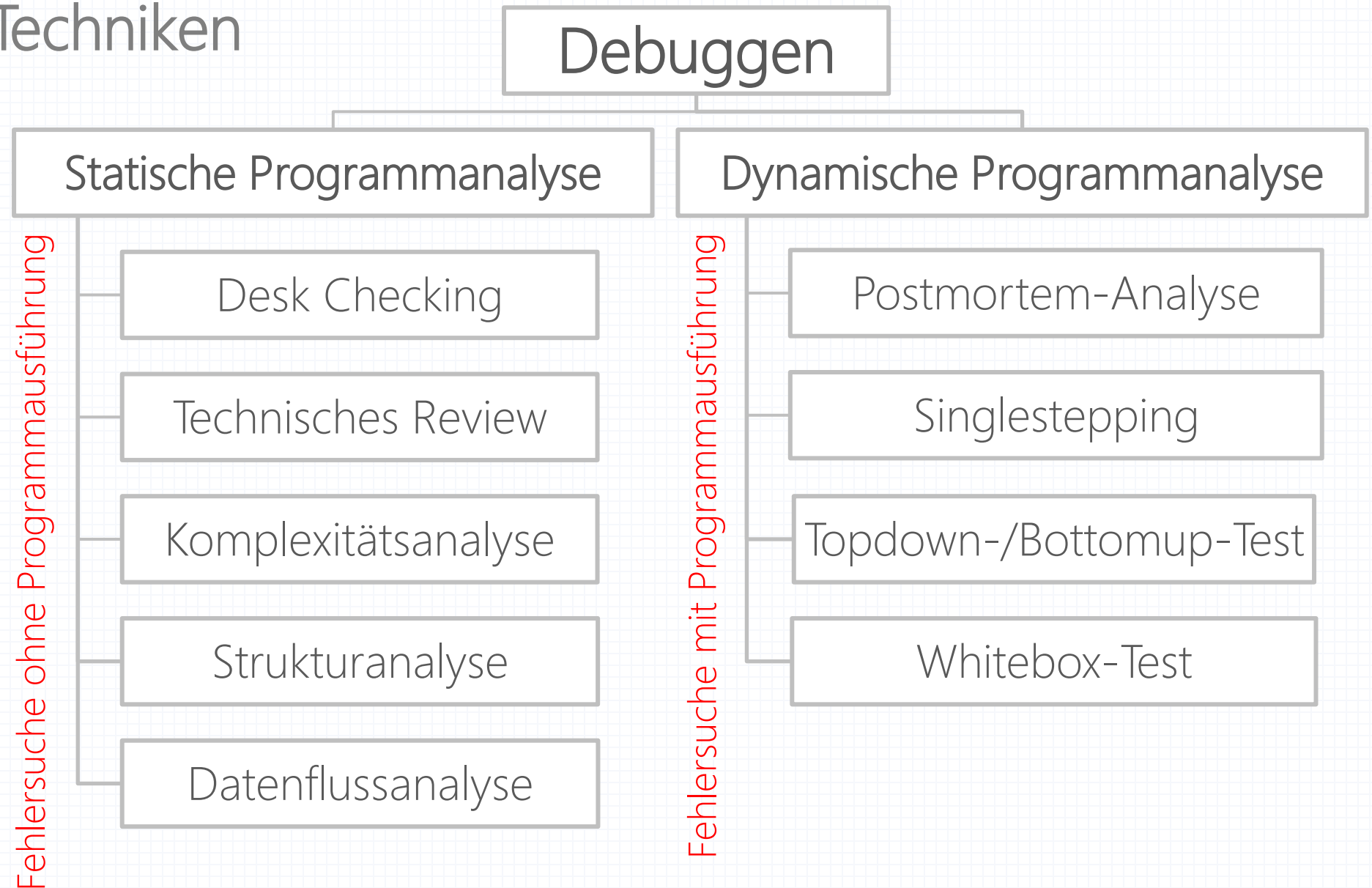
- andere Person testet
- Fehler nicht (genau) lokalisiert
- Fehler nicht behoben



(Grafik: geekandpoke.typepad.com)

Debuggen Techniken

Debuggen – Techniken



Techniken – Statische Programmanalyse (I)

Desk Checking („Schreibtischtest“)

manuelles Durchgehen „am Schreibtisch“,
oft anhand konkreter Beispiele



(Bild: www.stupidedia.org)

Aber: **Fachverständnis** (Domänenwissen) ist **wichtig!**

Beispiel/Übung: Fachverständnis – Wer kann schon Gälisch?

„Nid wyf yn y swyddfa ar hyn o bryd. Anfonwch unrhyw waith i'w gyfieithu.“



(Bild: bbc.co.uk)

Techniken – Statische Programmanalyse (II)

Technisches Review

Präsentation von Code in einer Gruppe („*First do reviews, then compile!*“)

Arten:

- **Walkthrough**: Implementierer präsentiert Programm Schritt für Schritt (Alternative: anderes Gruppenmitglied präsentiert)
- **Codeinspektion**: Gruppe prüft Code gegen (verschiedene) Checklisten
- **Codereview**: Gruppe prüft Code vorab und diskutiert ihn mit Implementierer

Zeitlimit ist wichtig!



(Bild: <http://www.jeetesh.net>)

Techniken – Statische Programmanalyse (III)

Komplexitätsanalyse

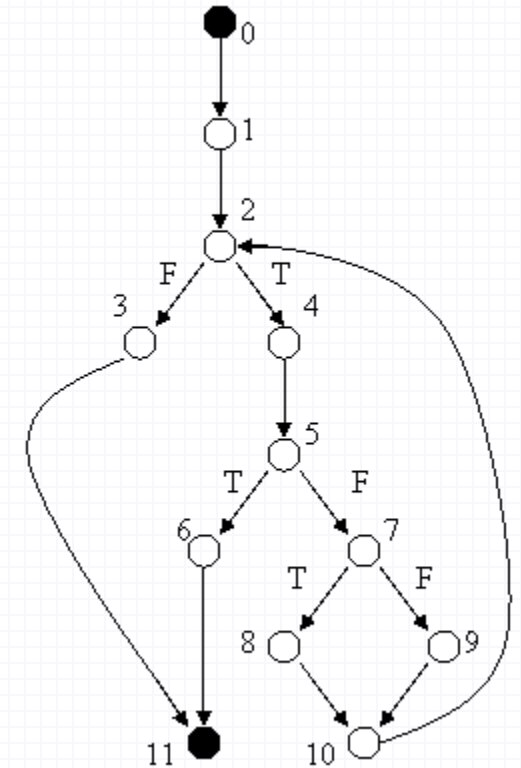
Bewertung mittels Komplexitätszahlen

Beispiele:

- Lines of Code (LOC)
- Software Sciences (Halstead)
- Cyclomatic Complexity (McCabe)

Viele Maße beruhen auf **keinem realistischen Modell**.
Es wird eher das gemessen, was leicht zu messen ist!

Aber: Eine schlechte Messung ist (meist) besser als gar keine!



(Grafik: whiteboxtest.com)

Techniken – Statische Programmanalyse (IV)

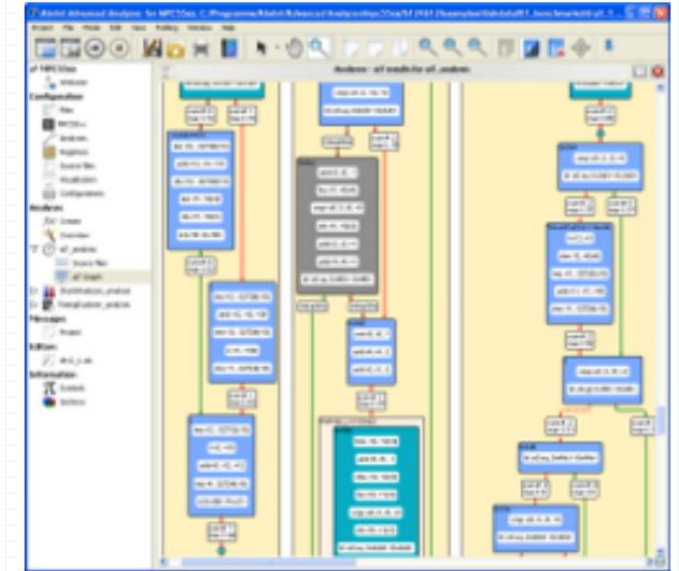
Strukturanalyse

Finden von Strukturanomalien
(z.B. unerreichbarem, „totem“ Code)

Datenflussanalyse

Finden von Datenflussanomalien
(z.B. verwendeten Variablen ohne Wert)

beides oft werkzeugunterstützt (z.B. *AbsInt*)



(Ausgegrauter „toter“ Code; Grafik: www.absint.com)



Techniken – Dynamische Programmanalyse

Postmortem-Analyse („Leichenbeschau“)

Auswerten der Ausgaben eines instrumentierten Codes (auch nach Absturz)

Singlestepping

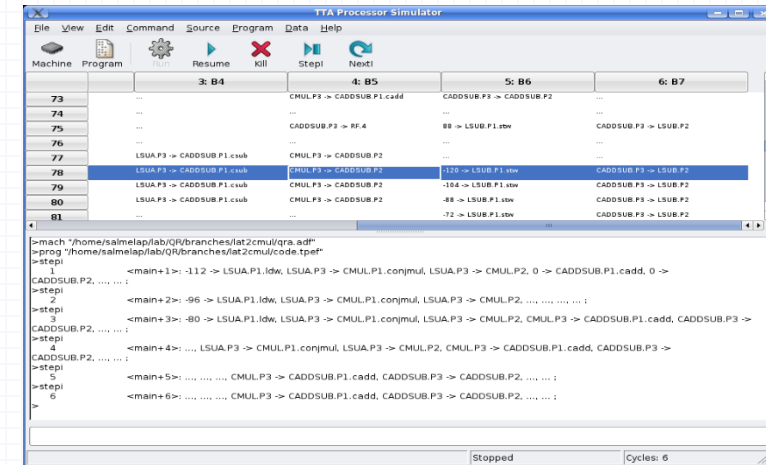
schrittweises Durchgehen eines Programms, z.B. mit **Debugger**

Topdown-/Bottomup-Test

Verwendung (von Teilen) des Testrahmens

Whitebox-Test

Programmlauf mit verfügbarem Quellcode (vielfach mittels Debugger)



Effizienz der Techniken beim Finden von Fehlern

aus einer Studie von Bob Grady (Hewlett-Packard, ~2000):

Testart	gefundene Fehler / h / Person
Reguläre Verwendung (Kunde)	0,21
Blackbox-Test	0,28
Whitebox-Test	0,32
Technisches Review	1,06

Beispiel/Übung: Effizienz der Techniken beim Finden von Fehlern

Wenn Sie eine der folgenden Techniken aus Kostengründen eliminieren müssten, **welche** würden Sie wählen und **warum**?

Testart	gefundene Fehler / h / Person
Reguläre Verwendung (Kunde)	0,21
Blackbox-Test	0,28
Whitebox-Test	0,32
Technisches Review	1,06

Debuggen Werkzeuge

Debuggen – Werkzeuge (I)

Statische Analysatoren

erstellen z. B. Objektbäume, Aufrufgraphen etc. zur Komplexitätsanalyse

Listengeneratoren

erstellen z. B. Listen von Komponenten, Aufrufen, Kreuzreferenzen, Programmstrukturpläne

Speicherprüfer

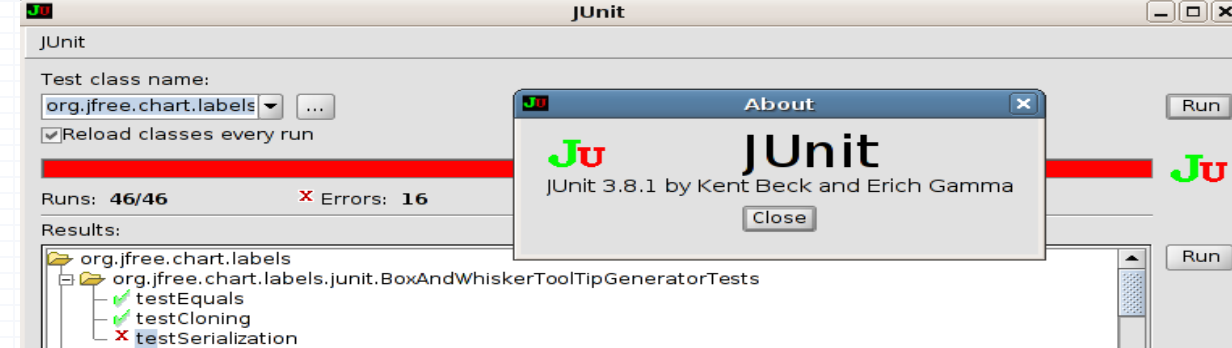
ermitteln verdächtige Speicherzugriffe

Debuggen – Werkzeuge (II)

Testrahmensysteme

Bibliotheken zur Testautomatisierung, die das Schreiben von Testtreibern vereinfachen und vereinheitlichen

- unterstützen ideal die **testgesteuerte Entwicklung**; analog dazu Unterteilung in
 - Testrahmensysteme für Systemtests
 - Testrahmensysteme für Komponententests (*Unit)
- benötigen sinnvollerweise Werkzeuge zur kontinuierlichen Integration
- ersetzen teilweise Debugging (Whitebox-Testen), aber keinesfalls das Blackbox-Testen
- bedeuten oftmals hohe Vorabinvestition (Zeit!) für – langfristig – bessere Qualität



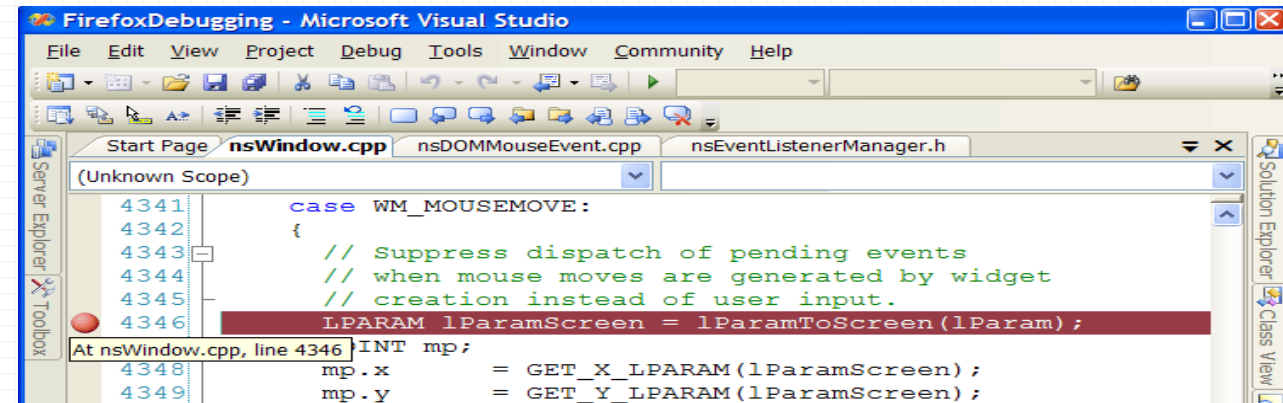
Debuggen – Werkzeuge (III)

Debugger

erlaubt Ablaufverfolgung während des Programmablaufs
(meist spezielle Übersetzung notwendig)

Funktionalität:

- Postmortem-Analyse
- Quellcode-Anzeige
- Zugriff auf Laufzeitwerte
- Darstellung von Aufrufketten
- Singlestepping, Setzen von Break Points und Watch Points
- Analyse dynamischer Objektstrukturen
- interaktives Ändern von Werten



Debuggen Ergebnisse

Debuggen – Ergebnisse (I)

Lauffähiges Programm

wird zum Testen (durch andere) freigegeben



(Grafik: www.dilbert.com)

Debuggen – Ergebnisse (II)

Fehlerliste

strukturierte Sammlung begangener oder möglicher Fehler
es gibt Standardlisten, jeder Softwareentwickler soll jedoch seine individuelle Fehlerliste anfertigen (gereiht nach Häufigkeit – Pareto-Prinzip)

Anwendung:

- zur strukturierten Fehlersuche
- zur Generierung von Lösungsideen
- zur Erstellung projektspezifischer Checklisten
- zur Schwerpunktsetzung bei knapper Testzeit



Ideale Musik-Kompilation für Softwareentwickler ;-)

(Idee: www.taraneon.de)

Scratching the surface – Saga
Money – Pink Floyd
Theme in search of a movie – Eddie Harris
Couldn't get it right – Climax Blues Band
Magna Of Illusion – Blue Oyster Cult
Rise – Herb Albert
Symphony No.8 (aka Unfinished) – Franz Schubert
Crawling from the wreckage – Rockpile
Waiting for an alibi – Thin Lizzy
Great Expectations – Miles Davis
Them Kinda Monkeys Can't Swing – Slade
Stop Breaking Down – UFO
How far Jerusalem – Magnum
Can I Have My Money Back? – Gerry Rafferty

